

DataSinking MCP Server

Expose the DataSinking Asian financial-report API to AI agents over Model Context Protocol (MCP) — no code required.

The endpoint

<https://api.datasink.ing/mcp>

Authorization: Bearer YOUR_KEY # or: https://api.datasink.ing/mcp?apikey=YOUR_KEY

Nothing to install. Prefer the header over ?apikey= where your client supports one — a key in a URL ends up in logs and screen shares.

How to add it, per client

Client	How
Claude Code	<code>claude mcp add --transport http datasinking https://api.datasink.ing/mcp --header "Authorization: Bearer YOUR_KEY"</code>
Claude Desktop	Settings → Connectors → Add custom connector, then paste the URL with ?apikey=YOUR_KEY already on it . A bare URL fails: the connector tries OAuth and this server has none
Cursor	<code>.cursor/mcp.json</code> (or <code>~/.cursor/mcp.json</code>) — a remote entry takes "url" + "headers", interpolating <code>\${env:DATASINK_API_KEY}</code>
OpenAI Codex CLI	<code>codex mcp add datasinking --url https://api.datasink.ing/mcp --bearer-token-env-var DATASINK_API_KEY</code>
WorkBuddy / CodeBuddy	插件 → MCP 服务器 → 配置 MCP, or edit <code>~/.workbuddy/mcp.json</code> / <code>~/.codebuddy/.mcp.json</code>

Client	How
Doubao Work / 豆包工作	技能 · 连接器 · 伙伴 → 新建自定义连接器 — HTTP transport, header Authorization: Bearer YOUR_KEY. No config file; desktop client only
Devin Desktop	%AppData%\devin\mcp_config.json on Windows, ~/.config/devin/mcp_config.json on macOS/Linux — the remote field is serverUrl, not url
DeepSeek Harness (dsh)	YAML patch layer at ~/.dsh/profiles/<name>/cordis.patch.yml, with the @deepseek-ai/dsh-mcp-client plugin

Each of these has a full guide — exact file, both scopes, verification, and the errors that client actually produces: docs/mcp/.

Tools

Tool	What it does	API
list_exchanges	List covered exchanges	/exchanges
list_stocks(exchange)	List stocks on an exchange	/stocks
list_reports(symbol, doc_type, size)	List a company' s reports (metadata)	/documents
get_report(document_id)	Get a report' s full text	/documents/{id}
list_sections(document_id)	List a report' s section headings	/documents/{id}/sections
get_section(document_id, section)	Get one section only	/documents/{id}?section=

Symbols are FMP-style: 600519.SS (Moutai), 005930.KS (Samsung), 7203.T (Toyota).

To save tokens, use get_section for one chapter — e.g. “management discussion and analysis” — instead of get_report for the whole document.

Running it locally instead

The same six tools, on your own machine, if you' d rather not send the key to a hosted endpoint:

```
npx -y datasinking-mcp # Node 18+
uvx --from "datasinking[mcp]" datasinking-mcp # Python, no install
```

The config shape is command + args + env:

```
{
  "mcpServers": {
    "datasinking": {
      "command": "npx",
      "args": ["-y", "datasinking-mcp"],
      "env": { "DATASINK_API_KEY": "YOUR_KEY" }
    }
  }
}
```

This JSON is a shape, not a drop-in. Every client's file differs in ways that break: Codex is TOML with no type field, Cursor interpolates `${env:NAME}` while Claude Code uses `${NAME}`, and Claude Desktop's file takes no url. Put it in the right file via that client's guide.

Whenever a config references the key instead of containing it — Codex's `bearer_token_env_var`, or `${DATASINK_API_KEY}` in a remote server's headers — set the variable **before** launching, or the header goes out empty:

```
export DATASINK_API_KEY="YOUR_KEY" # Linux / macOS
$env:DATASINK_API_KEY="YOUR_KEY" # Windows PowerShell
```

If it won't connect

Two properties of the hosted server cause most failures:

- **POST /mcp only.** There is no server-to-client SSE stream. With a valid key GET /mcp returns 405 Use POST /mcp for MCP streamable HTTP — but without one it returns 401 first, so a 401 from a client you configured correctly usually means it probed with GET.
- **Stateless.** No `Mcp-Session-Id` is issued or required, so nothing survives a restart and there is nothing to reset server-side.

Test the endpoint without any client involved:

```
curl -s -X POST https://api.datasink.ing/mcp \
-H "Authorization: Bearer YOUR_KEY" \
-H "Content-Type: application/json" \
-H "Accept: application/json, text/event-stream" \
-d '{"jsonrpc":"2.0","id":1,"method":"tools/list"}
```

Six tool definitions come back if the key is good. If it isn't, the body is the error — both HTTP 401:

Response	Meaning
<code>{"detail":"Missing API key"}</code>	Nothing was sent — the env var wasn't set before the client launched
<code>{"detail":"无效的 API key"}</code>	The key isn't recognised — check for a stray space or a missing Bearer

This one command separates “my client is misconfigured” from “my key is wrong” .

Keep the attribution

Every response carries a source field naming the official platform (cn-info.com.cn, EDINET, DART, MOPS). Keep it when you cite or redistribute the data.